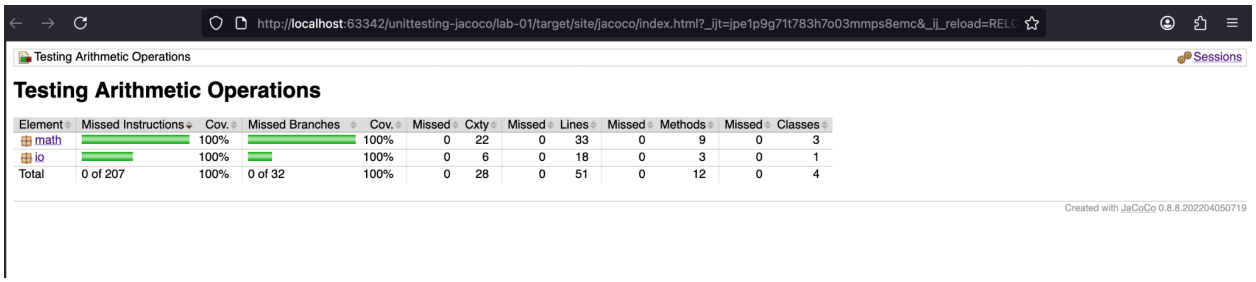
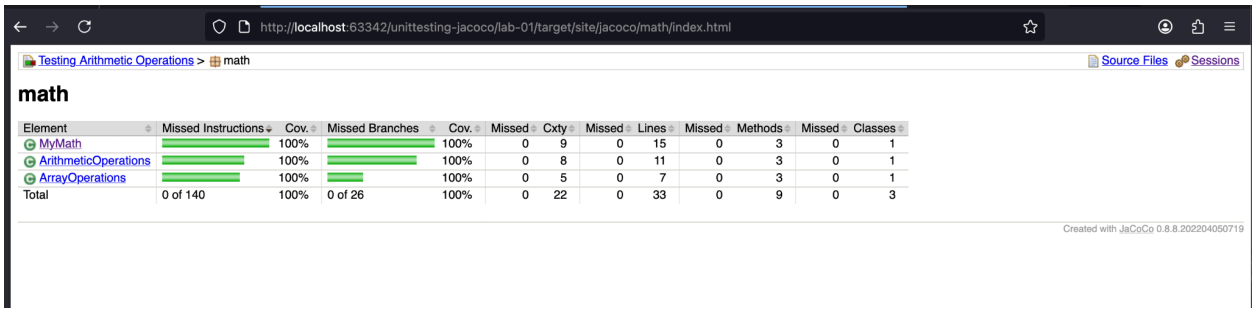
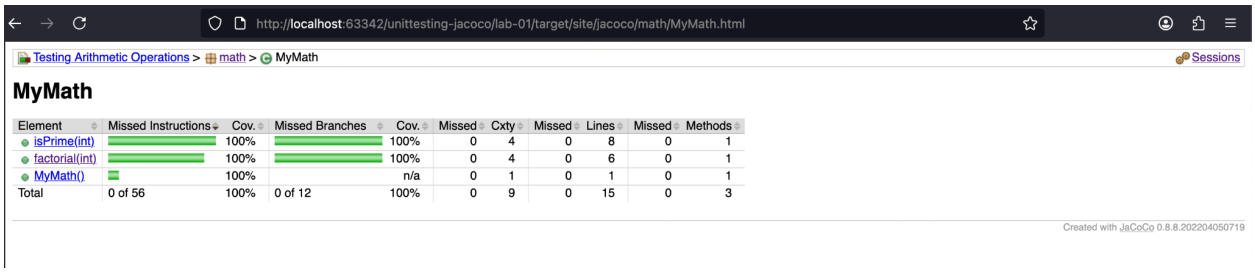
Overall statistics

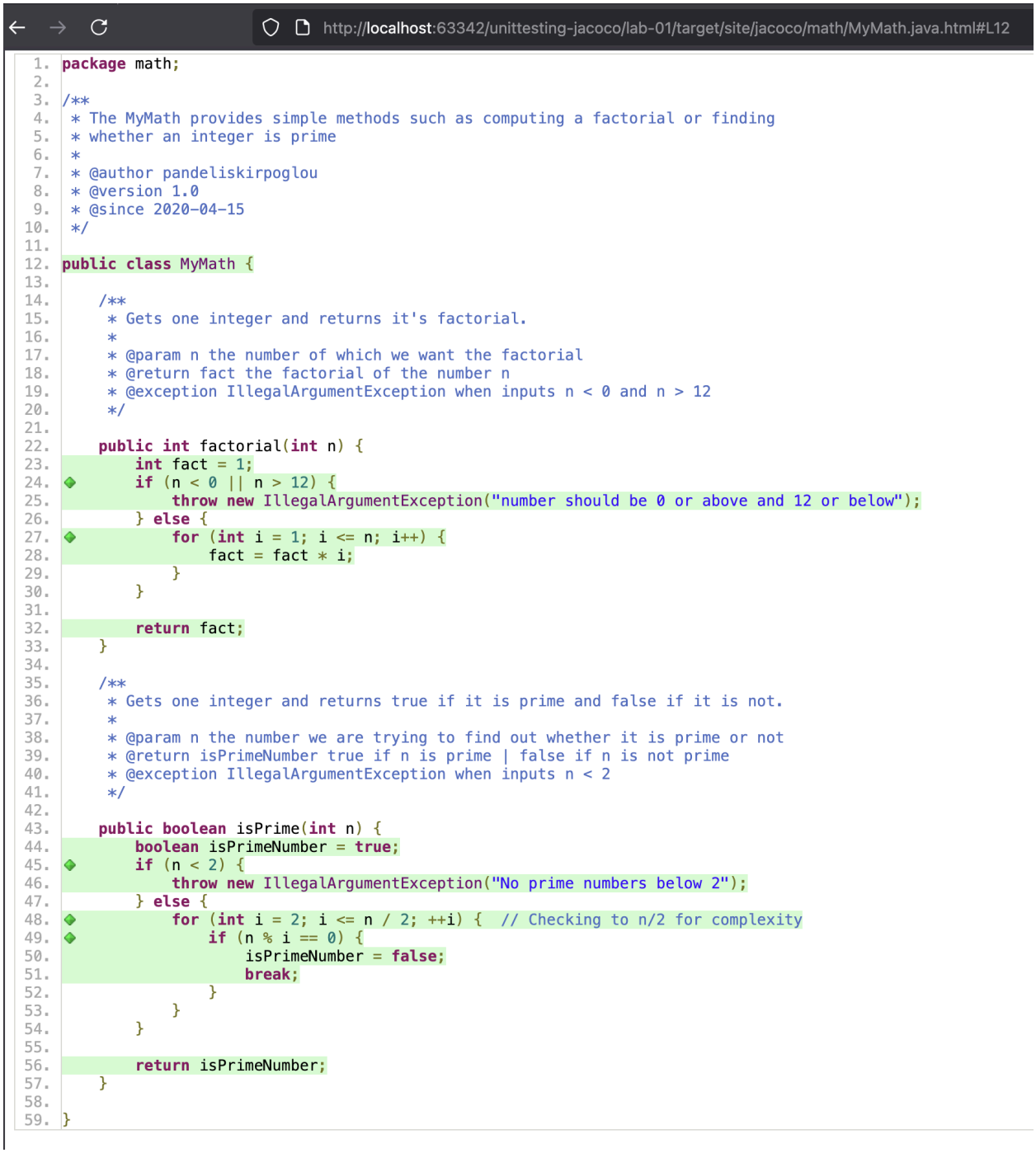


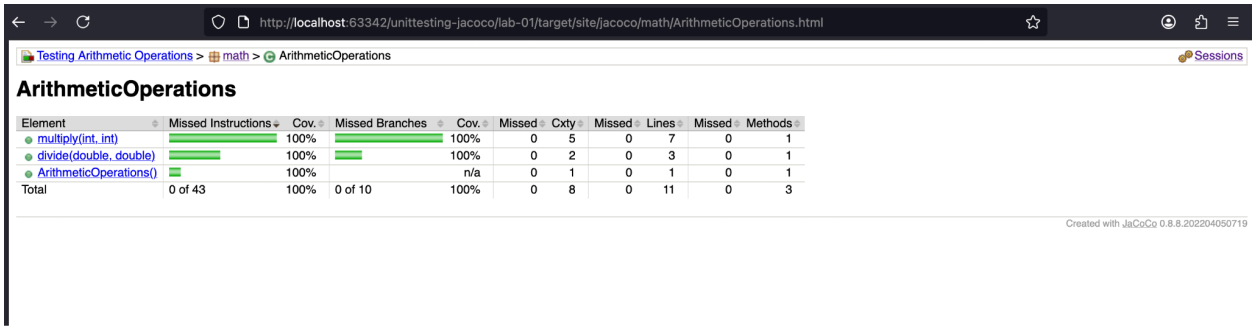
Package:- math

All coverages statistics




http://localhost:63342/unittesting-jacoco/lab-01/target/site/jacoco/math/MyMath.html
Testing Arithmetic Operations > math > MyMath
Sessions
MyMath
Element
Missed Instructions
Cov.
Missed Branches
Cov.
Missed
Cxty
Missed
Lines
Missed
Methods
isPrime(int) 100% 100% 0 4 0 8 0 1
factorial(int) 100% 100% 0 4 0 6 0 1
MyMath() 100% n/a 0 1 0 1 0 1
Total 0 of 56 100% 0 of 12 100% 0 9 0 15 0 3
Created with JaCoCo 0.8.8.202204050719


http://localhost:63342/unittesting-jacoco/lab-01/target/site/jacoco/math/MyMath.java.html#L12
1. package math;
2.
3. /**
4. * The MyMath provides simple methods such as computing a factorial or finding
5. * whether an integer is prime
6. *
7. * @author pandeliskirpoglou
8. * @version 1.0
9. * @since 2020-04-15
10. */
11.
12. public class MyMath {
13.
14. /**
15. * Gets one integer and returns it's factorial.
16. *
17. * @param n the number of which we want the factorial
18. * @return fact the factorial of the number n
19. * @exception IllegalArgumentException when inputs n < 0 and n > 12
20. */
21.
22. public int factorial(int n) {
23. int fact = 1;
24. if (n < 0 || n > 12) {
25. throw new IllegalArgumentException("number should be 0 or above and 12 or below");
26. } else {
27. for (int i = 1; i <= n; i++) {
28. fact = fact * i;
29. }
30. }
31.
32. return fact;
33. }
34.
35. /**
36. * Gets one integer and returns true if it is prime and false if it is not.
37. *
38. * @param n the number we are trying to find out whether it is prime or not
39. * @return isPrimeNumber true if n is prime | false if n is not prime
40. * @exception IllegalArgumentException when inputs n < 2
41. */
42.
43. public boolean isPrime(int n) {
44. boolean isPrimeNumber = true;
45. if (n < 2) {
46. throw new IllegalArgumentException("No prime numbers below 2");
47. } else {
48. for (int i = 2; i <= n / 2; ++i) { // Checking to n/2 for complexity
49. if (n % i == 0) {
50. isPrimeNumber = false;
51. break;
52. }
53. }
54. }
55.
56. return isPrimeNumber;
57. }
58.
59. }



http://localhost:63342/unittesting-jacoco/lab-01/target/site/jacoco/math/ArithmeticOperations.html

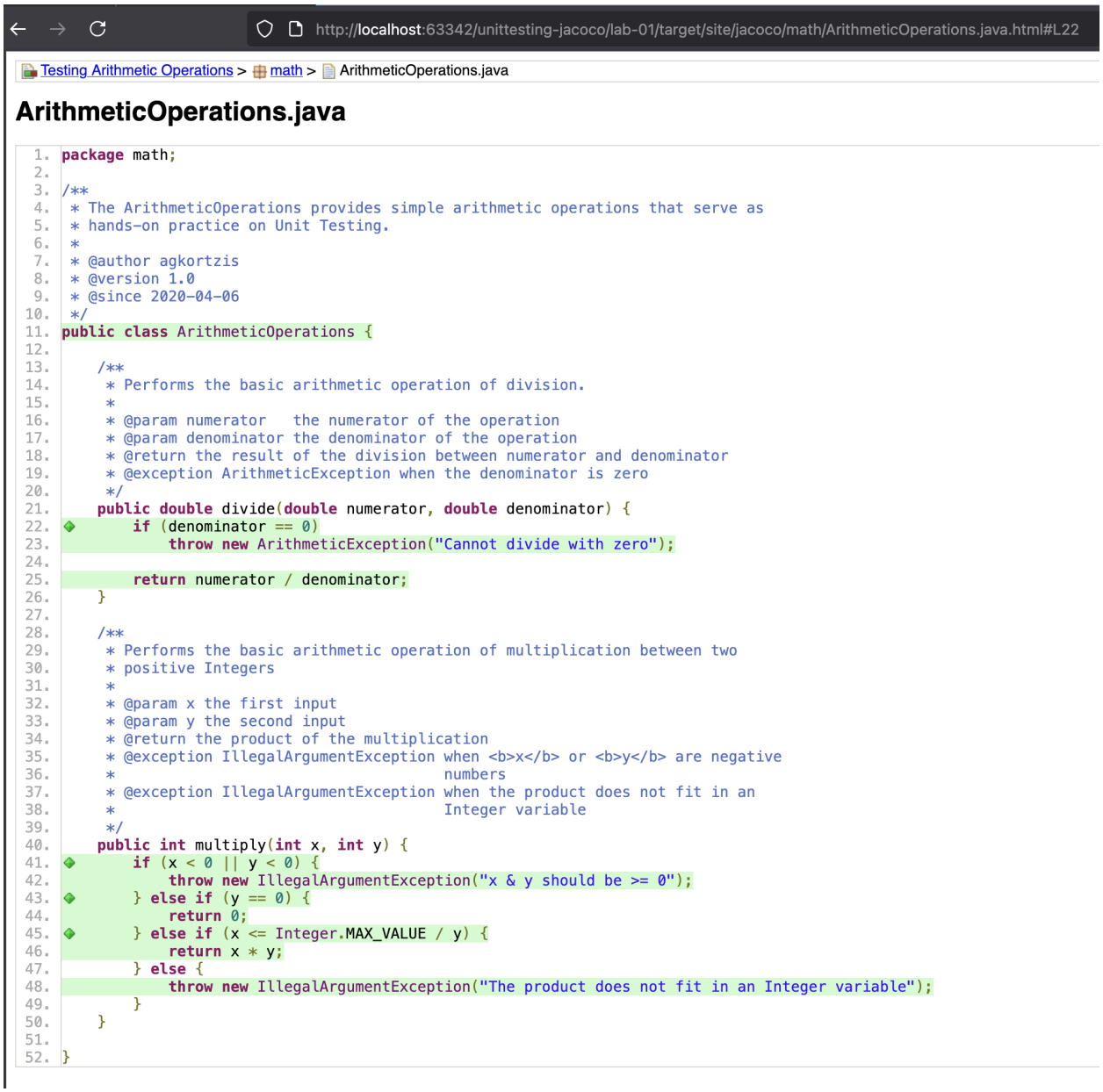
Testing Arithmetic Operations > math > ArithmeticOperations

Sessions

ArithmeticOperations

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
multiply(int, int)		100%		100%	0	5	0	7	0	1
divide(double, double)		100%		100%	0	2	0	3	0	1
ArithmeticOperations()		100%		n/a	0	1	0	1	0	1
Total	0 of 43	100%	0 of 10	100%	0	8	0	11	0	3

Created with JaCoCo 0.8.8.202204050719

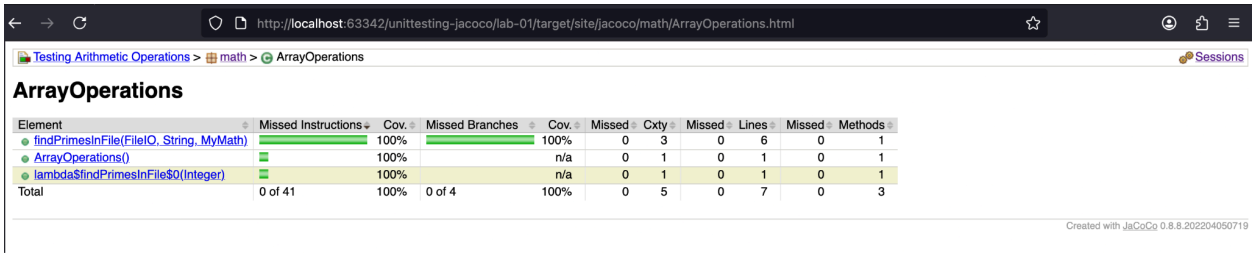


http://localhost:63342/unittesting-jacoco/lab-01/target/site/jacoco/math/ArithmeticOperations.java.html#L22

Testing Arithmetic Operations > math > ArithmeticOperations.java

ArithmeticOperations.java

```
1.  package math;
2.
3.  /**
4.   * The ArithmeticOperations provides simple arithmetic operations that serve as
5.   * hands-on practice on Unit Testing.
6.   *
7.   * @author agkortzis
8.   * @version 1.0
9.   * @since 2020-04-06
10.  */
11. public class ArithmeticOperations {
12.
13.     /**
14.      * Performs the basic arithmetic operation of division.
15.      *
16.      * @param numerator   the numerator of the operation
17.      * @param denominator the denominator of the operation
18.      * @return the result of the division between numerator and denominator
19.      * @exception ArithmeticException when the denominator is zero
20.      */
21.     public double divide(double numerator, double denominator) {
22.         if (denominator == 0)
23.             throw new ArithmeticException("Cannot divide with zero");
24.
25.         return numerator / denominator;
26.     }
27.
28.     /**
29.      * Performs the basic arithmetic operation of multiplication between two
30.      * positive Integers
31.      *
32.      * @param x the first input
33.      * @param y the second input
34.      * @return the product of the multiplication
35.      * @exception IllegalArgumentException when <b>x</b> or <b>y</b> are negative
36.      *                                     numbers
37.      * @exception IllegalArgumentException when the product does not fit in an
38.      *                                     Integer variable
39.      */
40.     public int multiply(int x, int y) {
41.         if (x < 0 || y < 0) {
42.             throw new IllegalArgumentException("x & y should be >= 0");
43.         } else if (y == 0) {
44.             return 0;
45.         } else if (x <= Integer.MAX_VALUE / y) {
46.             return x * y;
47.         } else {
48.             throw new IllegalArgumentException("The product does not fit in an Integer variable");
49.         }
50.     }
51.
52. }
```



http://localhost:63342/unittesting-jacoco/lab-01/target/site/jacoco/math/ArrayOperations.html

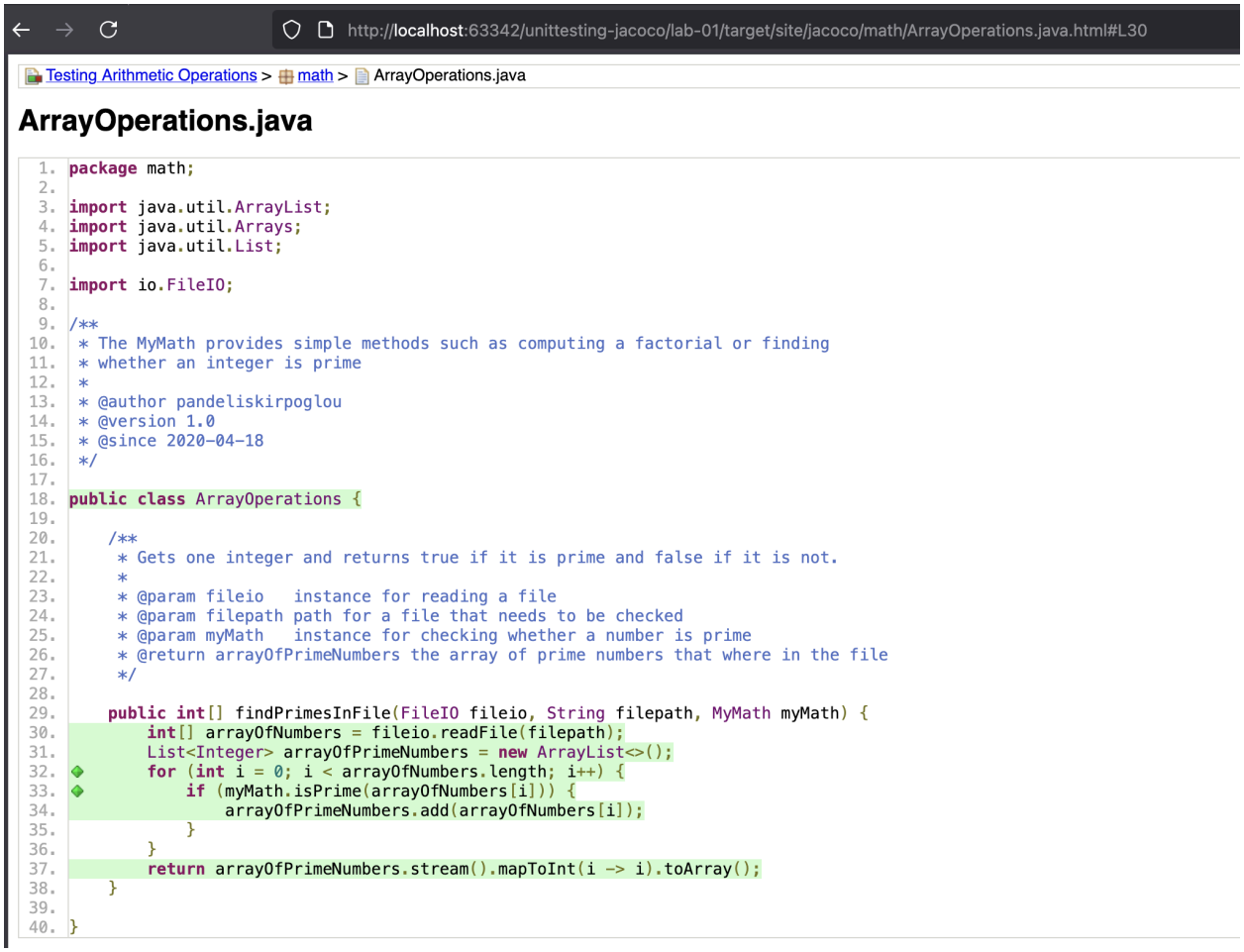
Testing Arithmetic Operations > math > ArrayOperations

Sessions

ArrayOperations

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
findPrimesInFile(FileIO, String, MyMath)		100%		100%	0	3	0	6	0	1
ArrayOperations()		100%		n/a	0	1	0	1	0	1
lambda$findPrimesInFile$0(Integer)		100%		n/a	0	1	0	1	0	1
Total	0 of 41	100%	0 of 4	100%	0	5	0	7	0	3

Created with JaCoCo 0.8.8.202204050719



http://localhost:63342/unittesting-jacoco/lab-01/target/site/jacoco/math/ArrayOperations.java.html#L30

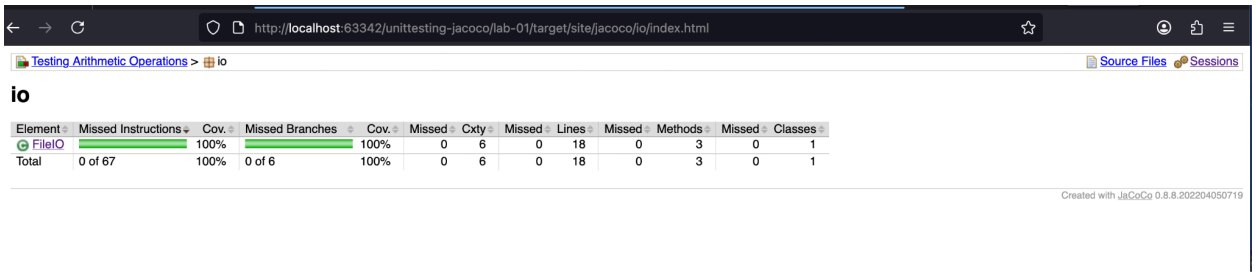
Testing Arithmetic Operations > math > ArrayOperations.java

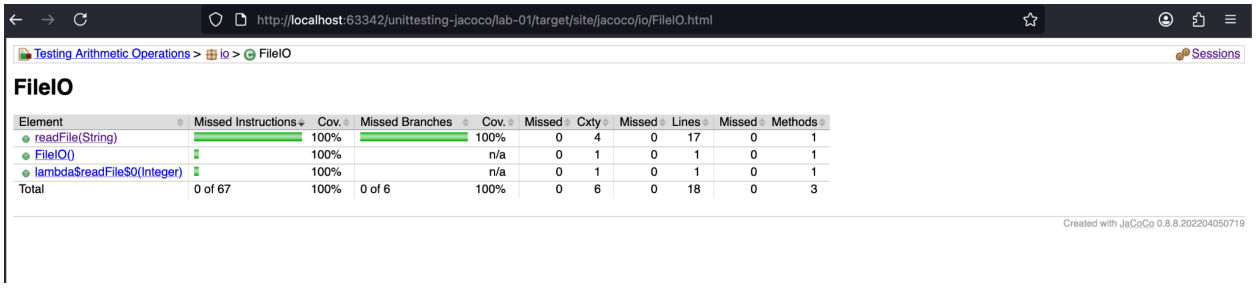
ArrayOperations.java

```
1.  package math;
2.
3.  import java.util.ArrayList;
4.  import java.util.Arrays;
5.  import java.util.List;
6.
7.  import io.FileIO;
8.
9.  /**
10.  * The MyMath provides simple methods such as computing a factorial or finding
11.  * whether an integer is prime
12.  *
13.  * @author pandeliskirpoglou
14.  * @version 1.0
15.  * @since 2020-04-18
16.  */
17.
18. public class ArrayOperations {
19.
20.     /**
21.      * Gets one integer and returns true if it is prime and false if it is not.
22.      *
23.      * @param fileio   instance for reading a file
24.      * @param filepath path for a file that needs to be checked
25.      * @param myMath   instance for checking whether a number is prime
26.      * @return arrayOfPrimeNumbers the array of prime numbers that where in the file
27.      */
28.
29.     public int[] findPrimesInFile(FileIO fileio, String filepath, MyMath myMath) {
30.         int[] arrayOfNumbers = fileio.readFile(filepath);
31.         List<Integer> arrayOfPrimeNumbers = new ArrayList<>();
32.         for (int i = 0; i < arrayOfNumbers.length; i++) {
33.             if (myMath.isPrime(arrayOfNumbers[i])) {
34.                 arrayOfPrimeNumbers.add(arrayOfNumbers[i]);
35.             }
36.         }
37.         return arrayOfPrimeNumbers.stream().mapToInt(i -> i).toArray();
38.     }
39.
40. }
```

Package:- io

All coverages statistics






http://localhost:63342/unittesting-jacoco/lab-01/target/site/jacoco/io/FileIO.java.html#L32
2.
3. import java.io.BufferedReader;
4. import java.io.File;
5. import java.io.FileNotFoundException;
6. import java.io.FileReader;
7. import java.io.IOException;
8. import java.util.ArrayList;
9. import java.util.List;
10.
11. /**
12. * The FileIO provides simple file input/output operations that serve as
13. * hands-on practice on Unit Testing.
14. *
15. * @author agkortzis
16. * @version 1.0
17. * @since 2020-04-06
18. */
19. public class FileIO {
20.
21. /**
22. * Reads a file that contains numbers line by line
23. * and returns an array of the integers found in the file.
24. * @param filepath the file that contains the numberses
25. * @return an array of numbers
26. * @exception IllegalArgumentException when the given file does not exist
27. * @exception IllegalArgumentException when the given file is empty
28. * @exception NumberFormatException for checking invalid entries
29. * @exception IOException when an IO interruption occurs (not required to be tested)
30. */
31. public int[] readFile(String filepath) {
32. File file = new File(filepath);
33. if (!file.exists())
34. throw new IllegalArgumentException("Input file does not exist");
35.
36. List<Integer> numbersList = new ArrayList<>();
37. BufferedReader reader;
38. try {
39. reader = new BufferedReader(new FileReader(file));
40. String line = null;
41. while ((line = reader.readLine()) != null) {
42. try {
43. int number = Integer.parseInt(line);
44. numbersList.add(number);
45. } catch (NumberFormatException e) {
46. // Do nothing will skip the the current invalid line
47. }
48. }
49. } catch (IOException e) {
50. e.printStackTrace();
51. }
52.
53. if (numbersList.size() == 0)
54. throw new IllegalArgumentException("Given file is empty");
55.
56. // Convert a List to an array using
57. return numbersList.stream().mapToInt(i -> i).toArray();
58. }
59.
60. }